Ch 12: Introduction to Recursion

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



Introduction to Recursion

Recursion

A recursive function is a function that calls itself.

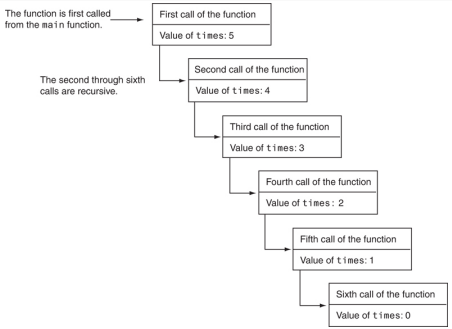


Figure 1: Six calls to the message function (Gaddis, 2021, pg.633)

- Recursive function **must** have a way to control the number of times it repeats
  - Usually involves an `if-else` statement which defines when the function should return a value and when it should call itself
- **Depth of recursion**: the number of times a function calls itself

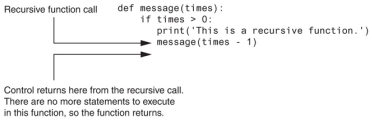


Figure 2: Control returns to the point after the recursive function call (Gaddis, 2021, pg.634)

EAST TEXAS A&M

Problem Solving with Recursion

Recursive Problem Solving

A problem can be solved with recursion if it can be broken down into smaller problems that are identical in structure to the overall problem.

- Recursion is a powerful tool for solving repetitive problems
- Recursion is never required to solve a problem
  - Any problem that can be solved recursively can be solved with a loop
  - Recursive algorithms usually less efficient than iterative ones
    - Due to overhead of each function call
- Some repetitive problems are more easily solved (or stated) with recursion
- General outline of a recursive function:
  - If the problem can be solved now without recursion, sole and return
    - Known as the **base case**
  - Otherwise, reduce problem to smaller problem(s) of the same structure and call the function again to solve the smaller problem
    - Known as the **recursive case**



Using Recursion to Calculate the Factorial of a Number (1 of 2)

- In mathematics, the $n!$ notation represents the factorial of a number n
  - For $n = 0$, $n! = 1$
  - For $n > 0$, $n! = 1 \times 2 \times 3 \cdots \times n$
  - For $n > 0$, $n! = n \times (n - 1)!$
- The above definition lends itself to recursive programming
  - $n = 0$ is the **base case**
  - $n > 0$ is the **recursive case**

```
factorial(n) = n x factorial(n - 1)
```

```
# The factorial function uses recursion
# to calculate the factorial of its
# argument, which is nonnegative.
def factorial(num):
    if num == 0:
        return 1
    else:
        return num * factorial(num - 1)
```

EAST TEXAS A&M

Using Recursion to Calculate the Factorial of a Number (2 of 2)

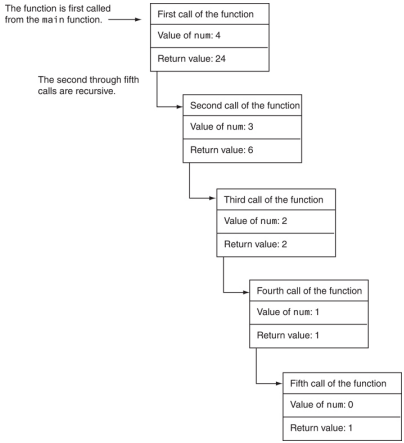


Figure 3: The value of num and the return value during each call of function (Gaddis, 2021, pg.637)

- Since each call to the recursive function reduces the problem:
  - Eventually it will get to the base case which does not require recursion, and the recursion will stop
- Usually the problem is reduced by making one or more parameters smaller at each function call

```
# The factorial function uses recursion
# to calculate the factorial of its
# argument, which is nonnegative.
def factorial(num):
    if num == 0:
        return 1
    else:
        return num * factorial(num - 1)
```

EAST TEXAS A&M

Direct and Indirect Recursion

- **Direct recursion**: when a function directly calls itself
  - All the examples shown so far were of direct recursion
- **Indirect recursion**: when function `A` calls function `B` which in turn calls function `A`

Examples of Recursive Algorithms

- Summing a range of list elements with recursion
  - Function receives a list containing range of elements to be summed, index of starting item in the range, and index of ending item in the range
  - Base case:
    - `if start > end return 0`
  - Recursive case:
    - `return list[start] + sum(list, start+1, end)`

```
# The range_sum function returns the sum
# of a range of items in num_list.  start
# specifies the index of the starting item.
# end specifies the index of the ending item.
def range_sum(num_list, start, end):
    if start > end:
        return 0
    else:
        return num_list[start] +
            range_sum(num_list, start+1, end)
```



The Fibonacci Series

- Fibonacci series: has two base cases
  - `if n = 0 then Fib(n) = 0`
  - `if n = 1 then Fib(n) = 1`
  - `if n > 1 then Fib(n) = Fib(n-1) + Fib(n-2)`

```
# The fib function returns the nth
# number in the Fibonacci series
def fib(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fib(n - 1) + fib(n - 2)
```

EAST TEXAS A&M

Finding the Greatest Common Divisor

- Calculation of the greatest common divisor (GCD) of two positive integers
  - If `x` can be evenly divided by `y` then `gcd(x,y) = y`
  - Otherwise, `gcd(x,y) = gcd(y, remainder of x/y)`

```
# The gcd function returns the greatest
# common divisor of two numbers.
def gcd(x, y):
    if x % y == 0:
        return y
    else:
        return gcd(x, x % y)
```

EAST TEXAS A&M

The Towers of Hanoi

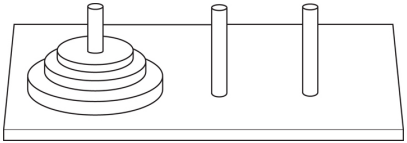


Figure 4: The pegs and discs in the Tower of Hanoi game (Gaddis, 2021, pg.642)

Game commonly used to illustrate recursion

- Three pegs and discs of decreasing size
- **Goal**: move the discs from leftmost to rightmost peg
  - Only one disc can be moved at a time
  - A disc cannot be placed on top of a smaller disc
  - All discs must be on a peg except while being moved

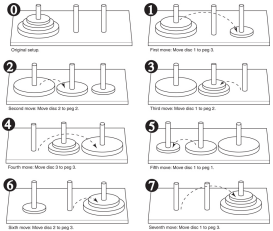


Figure 5: Steps moving three (Gaddis, 2021, pg.643)

Problem statement: move n discs from peg 1 to peg 3 using peg 2 as temporary peg

- if `n == 1`: Move disc from peg 1 to 3
- Otherwise:
  - Move `n - 1` discs from peg 1 to peg 2 using peg 3
  - Move remaining disc from peg 1 to peg 3
  - Move `n - 1` discs from peg 2 to peg 3

Recursion vs. Looping

- Reasons not to use recursion:
  - Less efficient, entails function calling overhead that is not necessary with a loop
  - Usually a solution using a loop is more evident than a recursive solution (?)
- Some problems are more easily solved with recursion than a loop
  - Example: Fibonacci, where the mathematical definition lends itself to recursion



Summary

- A recursive function is a function that calls itself
- Recursive function **must** have a way to control the number of times it repeats
  - **base case**: problem can be solved immediately, return solution
  - **recursive case**: reduce to a smaller problem(s) and call function again
- Recursion can be less efficient than a loop because of function call overhead
- On the other hand, certain problems are naturally defined in terms of recursive solutions
  - Especially in computer science



EAST TEXAS A&M

References

Gaddis, T. (2021). *Starting out with Python* (5th). Pearson.

EAST TEXAS A&M